

Data Driven and Machine Learning

Summary

The primary objective of this study was to develop surrogate models of the reactor, which must be able to provide similar results to the Computational Fluid Dynamics (CFD) model in a shorter period of time. The dataset was extracted from the CFD model in COMSOL, consisting of axial length, total inlet flowrate, coolant temperature, and the concentration of Propylene Glycol (PG). The extracted dataset was split into 80% for training and 20% for testing, with the PG concentration selected as the target. The machine learning (ML) models developed include Random Forest and Feed-Forward Neural Networks (MLP Regressor and TensorFlow). The Random Forest showed that the axial length is the most important feature, followed by the total flowrate and coolant temperature. The loss curves generated from the neural networks illustrated that the model only required a few iterations (Epochs) to stabilise. Hyperparameters were utilised and optimised for both models, and the result of only 1 hidden layer being required shows how simple the task is. The regression metrics and parity plot confirmed that all the ML models successfully learned the reactor function, providing accurate results. While the MLP Regressor proved to be the fastest model, the Random Forest was identified as the most suitable for this task, as it offers better stability and accuracy within an acceptable training time.

Dataset construction

5 different simulation cases were generated by varying the total inlet flowrate (Q_{in}) and inlet coolant temperature ($T_{in,c}$) of the base case. The dataset was generated in COMSOL using parametric sweeps and exported as a CSV file. The data was extracted along the central axis of the reactor ($r = 0$) along the axial length ($z = 0$ to $z = L$). A 'Regular Grid' export setting was used to export 100 data points for each case, resulting in a dataset of 500 rows in total. The specific operating conditions for the 5 simulation cases are:

- Case 1: $Q_{in} = 0.005 \text{ m}^3/\text{s}$ $T_{in,c} = 277 \text{ K}$ (**Base case**)
- Case 2: $Q_{in} = 0.005 \text{ m}^3/\text{s}$ $T_{in,c} = 267 \text{ K}$ (**10 K decrease**)
- Case 3: $Q_{in} = 0.005 \text{ m}^3/\text{s}$ $T_{in,c} = 288 \text{ K}$ (**11 K increase**)
- Case 4: $Q_{in} = 0.003 \text{ m}^3/\text{s}$ $T_{in,c} = 277 \text{ K}$ (**0.002 m³/s decrease**)
- Case 5: $Q_{in} = 0.007 \text{ m}^3/\text{s}$ $T_{in,c} = 277 \text{ K}$ (**0.002 m³/s increase**)

Inputs (features X): Axial position (z), total inlet flowrate (Q_{in}), inlet coolant temperature ($T_{in,c}$)

Output (target y): PG concentration (C_{PG})

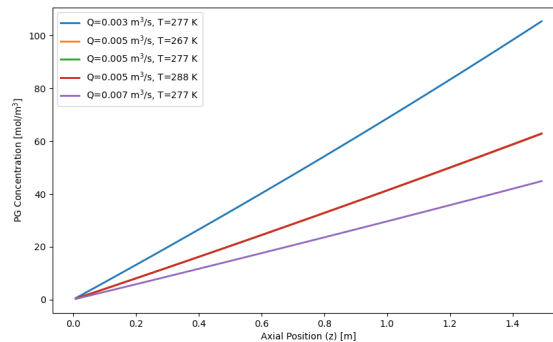


Figure 1. Plot of target variable (C_{PG}) along the reactor length (z)

The concentration plots for Cases 1-3 are nearly identical, as illustrated by the orange and green lines behind the red line in Figure 1. This demonstrates that the variations in coolant temperature effect have a minimal influence on the target variable compared to flowrate. This result is valuable for the validation of Machine Learning (ML) models performance, as this challenges the model's ability to distinguish between high impact feature (flowrate) and the lower impact feature (coolant temperature). The dataset was split randomly into 80% for training and 20% for testing, resulting in 400 rows of training data and 100 rows of test data.

Problem definition

Products and innovations are developed to solve problems, so does this machine learning tool. Fluid dynamics can be solved with the Navier stokes equations and CFD tools. But these methods require a lot of computational power and processing time, to solve spatiotemporal features via multiple partial differential equations, especially for the case of turbulent flow which has complicated eddies and fluctuations. The goal of this regression task is to learn the function $f(x)$ that predicts the target variable, (molar concentration of PG) based on the input variables consisting of the inlet flowrate, coolant temperature, and the axial position of the PFR.

The goal of this Machine Learning model is to be able to provide a similar outcome as the CFD model in a much shorter time, reducing the convergence and processing time from minute or hour to seconds. By being able to make prediction faster, engineers can now run more iterations in a short period of time, allowing them to be able to make design decisions responsively and optimise the design effectively by being able to see more outcome of the changes in design parameters. Same as optimisation, the tool can facilitate the control function as the observer can spot the system behaviour, which could lead to real time control. Moreover, this will also be much less expensive than the CFD software, allowing more students to access and learn more about the fluid flow, as Python is free.

However, the speed in prediction comes with a loss in accuracy. The accuracy may be affected by insufficient data, while the time required to train the machine can be very long. Consequently, accuracy and tractability of the model will have to be evaluated by comparing the performance with the CFD tool.

Baseline data processing

A naive baseline model was generated as a benchmark of the machine learning models. This simple regression mean model was created by calculating the mean of the target variable from the training dataset and applying this as a guess of the test data. As recommended in the brief, the Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 metrics were calculated to evaluate the quality of the guess as shown in Table 4, which will be utilised to assess the performance of the predictive machine learning models. The ML models must at least be better than this simple model (They must have smaller MAE and RMSE, and R^2 closer to 1).

Random Forest Regressor

A Random Forest model was trained to calculate the importance of each feature (input) to the target output through averaging the predictions of the decision trees. For this project, it was utilised to assess the importance of axial position, flowrate, and coolant temperature to the PG concentration.

The performance of a Random Forest is traditionally improved with hyperparameter tuning. The Grid Search tuning method was utilised to automatically tune the system, which tests the hyperparameter to find the best fit of the combinations. To ensure the model was not biased toward a single portion of data, Cross validation was also implemented. The hyperparameters below were automatically optimised with the goal to minimise MAE and RMSE, and get R^2 as close to 1.

Table 1. Random Forest optimised hyperparameter values

Hyperparameters	Values	Comments
n_estimators	800	Highest number of trees, indicating a large ensemble was required to stabilise the prediction.
max_depth	None	The model performed better by being detailed, (asking infinite questions).
max_features	sqrt	Prevents the tree from making dominant predictions, essential for this system with low and high influential features.
min_samples_split	2	Control tree growth, stops splitting if there is not enough data.
min_samples_leaf	1	Control overfitting, stops splitting at a small leaf size.
Bootstrap	True	Kept true to take only subset of columns and rows to train each decision tree.

The results of the optimised Random Forest model (MAE, RMSE, R^2) are summarised and compared in Figure 5 and Table 4. Figure 2 demonstrates a clear dominance of the axial position regarding

importance to the target, while the coolant temperature has almost a negligible effect. In addition, the importance score of the coolant temperature was even lower before the hyperparameter **max_features** was implemented.

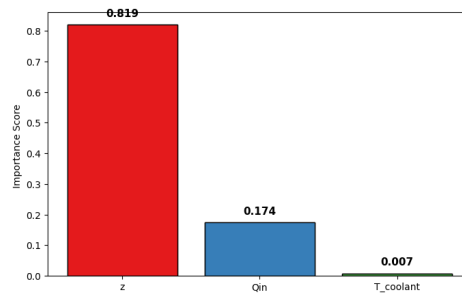


Figure 2. Importance bar chart

Feed Forward Neural Network

The Feed-Forward Neural Network (ANN) known as Multilayer Perception (MLP), models the non-linear regression relationships like the Random Forest. But it will be utilised to demonstrate how the model learns through validation of the loss curve, instead of feature importance. The feed-forward ANN or MLP consist of three layers: **input layer > hidden layer > output layer**, connected via weight node.

For this study, the feed forward ANN were modelled using TensorFlow and Scikit-Learn MLP Regressor, to compare the performance and computational requirement of both models. This is because MLP Regressor require less time to train than TensorFlow, hence, better for iterative works. It has to be noted that, while using the same dataset the TensorFlow was run on a different computer with a more powerful (RTX 2080ti), therefore the comparison in processing time will not be compared with another models.

Both were set up following the brief using ReLU activations, a linear output layer, Adam optimiser and Mean Squared Error (MSE). ReLU activations allow the model to learn non-linear relationships as the Navier Stokes and Arrhenius kinetics are extremely not linear. The data was scaled for standardisation as required. The GridSearchCV was also used to automatically tune the hidden layers, neuron per layer and learning rate as shown in Table 2. Both models chose the exact same hyperparameter values. The optimised result of only 1 hidden layer is in contrast with the interpretation of “Multilayer” and “Deep learning”. This is due to the simplicity of the reactor dimension or the fluid system itself. Therefore, this imply that 1 layer is enough and the use of ANN could be an overkill for this specific task.

Table 2. Neural Network optimised hyperparameter values

Tuning parameters	TensorFlow	MLP Regressor
Hidden layer	1	1
Neuron per layer	50	50
Learning rate	0.01	0.01
Epochs	500	-
Alpha	-	0.0001

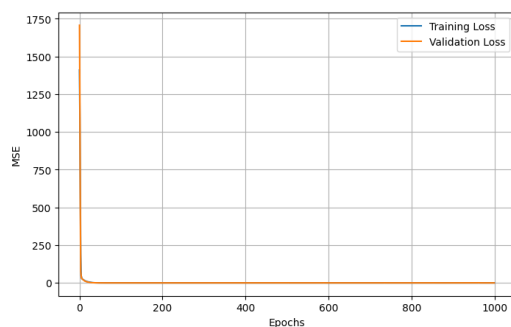


Figure 3. Loss curve (TensorFlow)

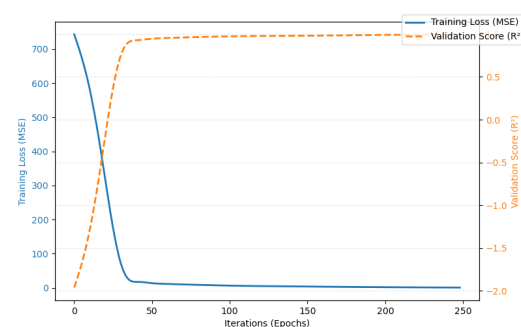


Figure 4. Loss curve (MLP Regressor)

The Loss curve generated from TensorFlow (Figure 3) is very insightful as it demonstrates a sharp drop of MSE at a few Epochs, which essentially mean that the Adam optimiser is very effective as it navigated

the loss almost immediately. As the training and loss curves are almost identical and show no divergence. This demonstrate that the model has successfully learn the model physics, instead of memorising the data. In addition, the inverse relationship between validation and loss in Figure 4 from the MLP regressor, also support that the system only requires a few Epochs to stabilise. Consequently, there is no sign of overfitting.

Model comparison and discussion

The regression metrics (MAE, RMSE, and R^2) of each model are compared as illustrated in Figure 5. All three models achieved exceptional results, with R^2 values approaching 1 (perfect fit), MAE values close to zero, and RMSE values aligning closely with the MAE. As shown in Table 4, all models surpassed the baseline significantly. The Random Forest model outperformed the MLP Regressor (MAE: 0.56 vs 0.84), concluding that the ensemble decision tree method is more suitable for this tabular dataset. It is a clear win for TensorFlow, it achieved the MAE of much lower than the other two models, and a R^2 of 1. This demonstrates that a high level of accuracy and precision of predictions are achievable when the deep learning run for longer iterations. However, TensorFlow failed to execute on the main AppleM3 computer and required a more powerful RTX 2080Ti to train, which still required longer training time than the other models (Table 3). Therefore, the computational and training time requirement is the limitation of this model, which making it viable only when negligible error (MAE<0.1) is required and if a powerful computer is available. Hence, the Random Forest and MLP Regressor are much more suitable for the iterative tasks that need fast responses, which aligns with the goal to beat the CFD's speed.

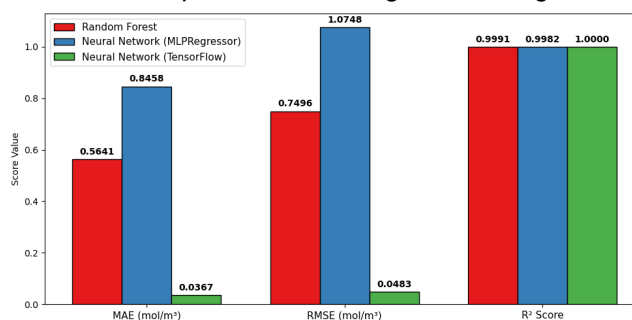


Figure 5. MAE, RMSE and R^2 of each regression models

Overall, these results confirm that the models have successfully learned the function $f(x)$. The parity plot (Figure 6) demonstrates the accuracy of both MLP Regressor and Random Forest, as both align closely to the perfect fit ($y=x$). However, while the parity plot presents the accuracy of both models, the residual plot in Figure 7 reveals that the Random Forest is much more stable, by having less outliers, especially near the reactor outlet. The parity and residual plots can be viewed in the **Appendix**.

Cost analysis (comparing CFD/COMSOL and ML)

The findings above concluded that both the Random Forest and Neural network models successfully learned the reactor function and were able to provide accurate and stable results comparable to the CFD. To assess the computational efficiency, the processing times (the time to run the code) of the ML models were compared with the time to run the CFD model. As shown in Table 3, the processing time of the base CFD model with the 'Fine' mesh setting in COMSOL is more than twice the time required to run the Random Forest model, while the MLP regressor is about 46 times faster than the CFD model. Given that they are confirmed to be highly accurate as shown in the parity plot, residual plot and the regression metrics (MAE, RMSE, R^2), both the Random Forest and MLP Regressor can effectively be used to replace the CFD results, depending on priority (accuracy, speed, or cost). If speed is the priority, MLP Regressor as the best option as it required the least training and prediction time. Overall, the Random Forest is concluded as the best choice for this task as it provides highly accurate and stable predictions in a short time, while TensorFlow is slower than the CFD, and MLP Regressor is less stable.

Table 3. Processing, training and prediction time comparison

	Random Forest	TensorFlow	MLP Regressor	CFD (Fine Mesh)
Processing time	102.2s	7 minutes (RTX 2080ti)	6.1s	281 s
Training time	0.5615 s	-	0.2399 s	-
Prediction time	0.0175 s	-	0.0001 s	-

Appendix

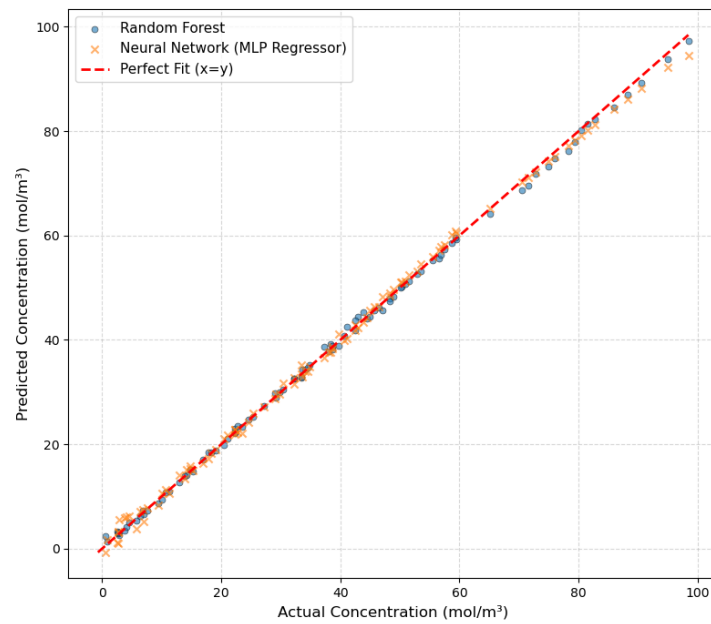


Figure 6. Parity plot predicted vs actual (both models)

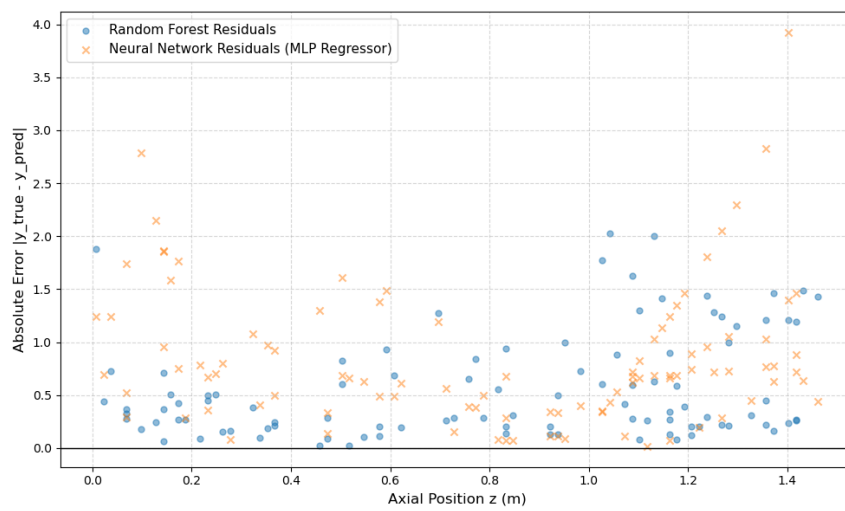


Figure 7. Residual plot

Table 4. MAE, RMSE and R^2 of each regression model

	Baseline	Random Forest	TensorFlow	MLP Regressor
MAE (mol/m ³)	20.6953	0.5641	0.0367	0.8458
RMSE (mol/m ³)	25.6355	0.7496	0.0483	1.0748
R^2 (-)	-0.0415	0.9991	1.0000	0.9982
Processing time	0s	1m 42.2s	7 minutes (RTX 2080ti)	6.1s